

ADRL in context

An independent reassessment of the research, the product opportunity, and the evidence still needed

7 September 2026 | [Research and architecture review](#) | [Decision support](#)

ADRL has a credible purpose and a promising architecture. Its strongest case today is controlling model-bound data and collecting trustworthy evidence. Its ability to reduce the total cost of real engineering work remains an experiment.

I agree with the earlier critique that routing needs better evidence, that model handoffs can be expensive, and that privacy controls must survive failures, retries and subagents. I disagree with several of its strongest conclusions. The papers do not establish that local-first is generally obsolete, that a learned policy must replace ADRL's current design, or that human approval of learned updates is behind the times.

Some disagreements are factual. A cited LiteLLM streaming bug was fixed in 2024. The implementation already signs learned-artifact manifests. A reported 15.3% routing improvement was attributed to the wrong comparison. Other disagreements concern what an experiment can establish outside its own setting.

The useful distinction is between three promises:

ADRL's promise	My assessment	Evidence still needed
Keep sensitive model traffic within approved boundaries	A strong architectural purpose; a bounded claim	Coverage, identity, bypass and failure tests on the actual deployment
Complete engineering work more cheaply	Plausible, with substantial uncertainty	Comparable task outcomes, actual spend, waiting time and repair effort
Learn which model works best for this organization	A sensible direction, still early	Representative verified outcomes and trustworthy comparisons with alternatives

This is my synthesis of the register, implementation excerpts and primary sources, not a measured ranking against commercial products. The supplied product report describes a D1/D2 implementation with no organic traffic; this review did not establish a later production result. Existing prototype maturity should not be transferred to the new build.

My recommendation is to continue with a narrow, instrumented pilot. Preserve the controls that constrain what learning may do. Correct the research overstatements. Spend the next unit of effort obtaining evidence about one useful workload before broadening harness support or adding a more sophisticated router.

For a reader with limited time, pages 2-4 explain the landscape and taxonomy; pages 8-9 give my disposition of all fourteen ranked findings. The final pages describe the experiments, source checks and evidence behind this assessment.

Where ADRL belongs in the landscape

A coding agent combines several systems. The harness decides how to interact with files and tools. A model produces the next step. A gateway delivers model requests. Security controls decide which actions and destinations are allowed. An evaluation system determines whether the work actually helped.

ADRL sits at the intersection of these systems. That position is useful because a routing decision is also a data-placement decision: choosing a cloud model sends it the context, while choosing a model on the machine can keep that inference local.

Neighboring system	Main job	ADRL's relationship to it
Model gateway	Deliver requests, manage credentials, retries and endpoints	ADRL adds policy before dispatch and constrains the gateway's choices
Model router	Choose a model for a task or stage	ADRL can host a router, but must also honor privacy and protocol state
Coding harness	Run tools, manage conversation and workspace state	ADRL depends on its signals; an HTTP proxy does not own every tool action
Agent security runtime	Mediate tools and sensitive information flows	ADRL provides narrower model-egress controls unless integrated more deeply
Evaluation and learning system	Measure results and improve future decisions	ADRL's ledger can connect policy, execution and verified outcomes

These are roles, not mutually exclusive product categories. A gateway can gain routing features; a harness can gain privacy controls. LiteLLM already documents proxy access to local Ollama models, so a local target by itself is not a defensible uniqueness claim. The stronger proposition is applying an organization's policy before model context leaves the laptop, carrying that policy across the conversation, and retaining evidence of the result. [D5](#)

The research landscape is moving from selecting a model from the opening prompt toward using information revealed during work. It also contains substantial earlier routing research: RouteLLM already learned from comparative outcomes, while the unified routing-and-cascading work studied when trying another model helps. ADRL assembles ideas with precedents; the sources reviewed do not establish a novel routing algorithm. [S1](#), [S2](#)

Consider a developer fixing a failing payment test. Before the first model call, repository classification may already prohibit cloud inference. Later, a file read might reveal a credential. If cloud use remains permitted, ADRL can ask whether the cheaper model is making verifiable progress. If a switch is worthwhile, it must preserve tool state and record what actually served the request.

Those are separate decisions. A model being cheaper does not make its destination permissible. A destination being permissible does not mean that model can finish the job. Keeping those questions separate is one of ADRL's most valuable design choices.

The taxonomy: a useful map of responsibility

I agree with the architectural taxonomy. Its value is making responsibilities and dependencies explicit. The number of buckets does not establish scientific novelty or product readiness. This assessment includes a direct reading of the dedicated overview, the current index and the TRU bucket rationale.

The original overview has nine buckets. The 3 September register adds **TRU: Trust, Residency and Egress**, making ten. The product report describes TRU but still says nine. This is document drift: the newer structure separates the identity and destination questions from content scanning and blocking. That separation is useful and should appear consistently in the product explanation.

Bucket	In ordinary language	My assessment
FND: Foundations	Set the system's rules and boundaries	Keep; make every product promise fit the boundary
SEM: Interaction semantics	Understand what this interaction means	Essential; compatibility depends on actual harness traffic
TRU: Trust and egress	Establish whose work this is and allowed destinations	Keep separate; a model tier is not a security boundary
SAF: Safety and constraints	Apply restrictions before choosing a model	Keep; measure both missed restrictions and unnecessary blocks
RTG: Routing	Choose among permitted options	Economically promising; advantage remains unproven
CAS: Execution and recovery	Execute, switch or stop without corrupting work	Essential; test handoffs and side effects together
MEM: Evidence and memory	Record what happened and how well it is known	A strong foundation; verification and erasure need care
LRN: Learning	Learn from trustworthy comparisons	Keep gated; let competing algorithms earn authority
EVL: Evaluation	Decide what evidence warrants wider use	Essential; separate working machinery from demonstrated benefit
OPS: Operations	Keep controls working and recover from failure	Essential; test silent failures as well as visible errors

The request flow is **understand the interaction, establish identity and allowed destinations, apply gates, choose, then execute**. The evidence flow is **record the decision and outcome, verify it, evaluate a candidate policy, then approve any wider authority**. FND and OPS support both. TRU participates before dispatch and in the later destination record; it is not merely an audit step at the end.

This map places ADRL across routing, information-flow controls and evaluation infrastructure. Its most convincing contribution would be making those responsibilities work together in an actual deployment. Evidence for one component should not be borrowed to certify the entire chain.

Four kinds of classification, four different jobs

The documents also use taxonomy for labels inside the system. Those labels answer different questions and should not become interchangeable inputs to a single judgment of "safe and easy."

Classification	What it tells ADRL	What it cannot establish alone
Architectural bucket	Which component owns a responsibility	Whether that component works in production
Request class, SEM-001	Turn, continuation, utility, subagent, pre-warm or passthrough	Whether its payload may leave the machine
Proposed SDLC intent	The developer's activity, such as fixing code or writing tests	Task difficulty, data sensitivity or the best model
Failure type, MEM/CAS	A reason attributed to an unsuccessful outcome	Proven causality or permission to train on that label

The proposed task taxonomy is a hypothesis about useful features. The overview names `sdlc-intent-v1` and `modernbert-sdlc-v1`, with examples `change.fix` and `assure.test`, and describes them as unbuilt. I found that proposal, but not a complete label specification in the inspected materials. This review therefore assesses its role; it does not validate an unseen label set or classifier.

For example, "fix a typo" and "fix a concurrency bug in settlement" can both be fixes while needing very different capabilities and safeguards. A task label may help select evaluation slices or retrieve comparable experience. It should earn influence on routing by improving measured choices beyond existing features. The routing literature reviewed here supports testing informative features; it does not validate this particular taxonomy. [S3](#), [S4](#)

My recommendation is to version the label definitions, preserve classifier version and confidence, allow unknown or multiple intents, and retain corrections. Evaluate classification reliability separately from routing benefit. A useful classifier can still add no economic value if all models perform similarly on the groups it distinguishes.

Request classes need a different discipline. A token-count or compaction interaction can contain sensitive context even though it is housekeeping. SEM-001 already recognizes that content-bearing distinction. Preserve it, add compatibility fixtures for evolving compaction and permission-classifier behavior, and keep workload authorization independent of text that the model or repository can influence. [D1](#), [D3](#)

Maturity is another independent axis. An accepted design can remain D0. D3 shadow operation can establish observation and plumbing without proving a cheaper executed policy. The overview explicitly separates architectural agreement from D0-D5 maturity and states limits on its single-user evidence. I agree with that distinction. A literature verdict of CURRENT must never be presented as a readiness certificate.

What the routing science actually says

A router needs information that changes the choice, not simply a more elaborate classifier. The Routing Plateau compares 21 methods across five benchmarks and finds that many perform similarly. But its scope is single-shot routing over fixed model pools; the authors explicitly exclude cascades and changing pools. They also demonstrate improvements from more data, stronger encoders and fine-tuning. This supports skepticism about easy routing gains, not a universal impossibility result. [S3](#)

SWE-Router makes an intuitive proposal: let a cheap model inspect the problem briefly, then decide whether to continue or escalate using what it learned. Its idealized result says a decision-maker can benefit from having more information available. It does not guarantee that a finite learned router will interpret that information correctly, or that obtaining it pays for its latency and side effects. [S4](#)

A handoff changes more than the model name. The Handoff Tax studies two model pairs, 500 SWE-bench Verified tasks and 58,000 runs. Its findings make inherited context a serious design concern. However, switch points are fixed in advance, there is one rollout per task/configuration, and dollar-cost conclusions depend on pricing and cache behavior. It does not test ADRL's exact local models, privacy constraints or adaptive trip-wires. [S5](#)

The Replay Gap supplies a different warning: if a replacement model takes different actions, scoring it against the original model's later observations can evaluate an imaginary execution. Its pilot uses 30 instances per run pair, two quantized models, a 28k context budget and only 0-3% task resolution. The paper itself calls its directional routing recommendations hypotheses. Its strongest contribution here is evaluation discipline, not proof that escalation is economically backwards. [S6](#)

The practical implication is to compare several policies under the same conditions:

- Start with the chosen strong model and stay there.
- Start cheaply, with a bounded attempt and verified escalation signals.
- Use a short information-gathering phase before choosing the completion model.
- Start strongly, then reduce cost at a tested boundary.

All comparisons must honor the same permitted destinations. Privacy-pinned work is a separate population: cloud escalation is unavailable, regardless of its benchmark performance.

ADRL's RTG-004 already makes local-first conditional on a bounded attempt, a feasible higher destination and recoverable side effects. RTG-002 already charges expected cascade cost. Those provisions deserve testing and better estimates; they should not be described as unconditional faith in the cheapest model.

The quantity to optimize is **the cost of getting acceptable work completed**. Token spend is one component. Repeated attempts, handoffs, verification, developer waiting and later repair can change which policy wins. A lower API bill is a useful result only when its quality and time consequences are visible.

Learning from experience: promising, but easy to misread

There is a meaningful scientific case for using verified past outcomes. There is much less support for removing ADRL's evaluation and approval requirements.

Agent-as-a-Router reports that adding task-level performance statistics raises a vanilla router's score from 41.41 to 47.74: a 15.3% relative improvement. The heuristic using comparable statistics scores 47.50. The improvement over that heuristic is therefore 0.24 score points, approximately 0.5% relative, not 15.3%. This correction concerns the preliminary ablation; it does not erase the paper's separate results for its full system. The authors also qualify their cost comparison because cache hit rates are not observed. [S7](#)

MEM-008 says retrieval stays advisory **until** evaluation gates pass. That is compatible with eventually learning from verified experience. Treating the decision as a permanent ban creates a disagreement that the text does not contain.

BaRP provides a legitimate alternative to an effect estimator: learn a policy whose choices can change with the operator's cost-quality preference. This challenges LRN-003's absolute ban on outputs that directly select a model. However, BaRP trains on static offline logs with simulated partial feedback, and its formulation is single-step. It is not a demonstration that autonomous updates are safe in an enterprise coding deployment. [S8](#)

I would retain explicit graduation under LRN-007. I would make LRN-003 less prescriptive about the algorithm: require outcomes-based training, evaluation under changing costs, uncertainty or abstention, and enforceable privacy constraints. Let a direct policy compete with an effect estimator under the same test.

The evidence rules also need a correction before training. LRN-001 limits the objective to organic T1 labels and puts counterfactual pairs in a separate T5 tier. LRN-002 and LRN-003 make those pairs central training evidence. These instructions need reconciliation. Record two properties separately: where evidence came from, and how well its outcome was verified. Then explicitly state which combinations may train, calibrate or evaluate each component. This is my architectural finding, not a result imported from a paper.

Finally, a green test result is an observation about a particular test suite. It is not complete proof that a task was solved well. SlopCodeBench studies repeated extension of prior code and finds quality deterioration that one-shot success measures miss. ADRL should retain its deterministic checks while also measuring reverts, corrective follow-ups and reviewer repair on a sampled, clearly labeled basis. Those observations should not silently become perfect training labels. [S12](#)

The learning opportunity is real. Its scarce input is a trustworthy comparison of what would have happened under another permitted choice.

Privacy and security need a clear boundary

I agree with hard gates before optimization, durable restrictions and explicit release. Security should not depend on the same model deciding that its own action is safe. CaMeL and information-flow-control research support separating untrusted content from authority and checking how information reaches tools and recipients. They also require substantial participation from the runtime and its tools. An API proxy does not inherit their guarantees simply by using similar language. [S13](#), [S14](#)

For ADRL, four distinct questions must remain visible:

Question	What can answer it
Was this content permitted to reach this model?	Authenticated policy, gates and the permitted deployment set
Did the gateway report the expected destination?	A response receipt, with its provenance and uncertainty
Could tools send data through another path?	Harness and operating-system controls, with explicit coverage
Was the audit history subsequently altered?	Integrity checks and independently retained checkpoints

A receipt is detective evidence: it arrives after dispatch. A signed log protects recorded history; it does not prove the completeness or truth of every event recorded. These are engineering distinctions derived from ADRL's control boundaries, not claims that those mechanisms are useless.

Subagents illustrate the boundary well. If a child reads a secret and sends the raw result to its parent, the parent's next cloud request now contains that secret. Parent-to-child pin inheritance alone cannot address that return path. Either block the return, propagate its restriction before the next dispatch, or use an explicitly trusted sanitization mechanism. APPA v2 demonstrates useful recovery through controlled branching, but assumes trusted contracts, monitor and sanitizer code. Its zero observed attacks in the tested setting are not a general guarantee. [S15](#)

The earlier critique is also right to question a detector's blind spots, but an LLM is not the mandatory next detector. Research supports hybrid regex-plus-LLM classification, while another recent study explores string-based detection. Neither source establishes performance on ADRL's live tool results. Compare candidate detectors locally on representative data, measuring missed secrets, false pins, latency and unsupported content. Preserve repository-level restrictions so policy does not wait for a recognizable secret. [S16](#), [S17](#)

For storage, embeddings should remain sensitive. Ghost Vectors shows that soft deletion can leave recoverable vectors in tested HNSW systems, but reconstruction quality varies sharply by data type. That is not proof that every rebuilt ADRL index leaks. Zero2Text supplies a separate reason not to treat an embedding as anonymization. Test the actual files, backups and key lifecycle. [S18](#), [S19](#)

EDPB's blockchain guidance supports considering deletable payload storage; applying it to ADRL requires separate analysis. Anthropic recommends a dedicated VM for untrusted repositories. Neither source makes all ledgers or isolation mechanisms equivalent. [D6](#), [D7](#)

My disposition of the fourteen ranked findings

The following verdicts concern the earlier critique's proposed conclusions. "Qualified agreement" means the underlying concern is useful but its scope or remedy needs revision. These are recommendations for disposition, not edits to the register.

1. Escalation economics have inverted: qualified agreement. Handoff direction and inherited context matter. Test compact handoffs and downshifting. Do not replace conditional local-first on the strength of these studies alone. The two central papers study different questions and have different limitations. A shadow decision alone cannot measure the result of a switch that never happened. Affected: RTG-004, CAS-004/005. [S5](#), [S6](#)

2. Only trajectory-conditioned routing escapes the plateau: disagree with the absolute claim. Add progress-aware candidates to the experiment, but retain strong simple baselines. The plateau paper itself studies other improvement paths. A Bayes-optimal result about available information is not a deployment guarantee. Affected: RTG-002/003/006/007. [S3](#), [S4](#)

3. Outcome memory and direct policies require weakening advisory and graduation rules: disagree with that inference. Correct the 15.3% comparison. Keep MEM-008's evidence gate and LRN-007's approval boundary. Allow alternative policy representations to compete rather than mandating a CATE model. Affected: MEM-008, LRN-003/007. [S7](#), [S8](#)

4. Thinking signatures make handoff a security and compatibility boundary: agree on the protocol change. Current Anthropic documentation binds thinking to model and prefix under stated conditions. Validate transformations per model and API version, preserve tool state, and log transformations. The earlier incident's extracted-key and trace counts were not independently established in this review and are not needed to justify this amendment. Affected: CAS-004/006, RTG-008. [D2](#)

5. Compaction and safety-classifier traffic require semantic updates: agree. Server-side compaction can occur inside a response rather than as a standalone utility request. Safety-classifier traffic must not be casually rerouted to change the harness's safety decision. Identity markers themselves vary by configuration and release. Maintain captured compatibility fixtures instead of a permanent shape assumption. Affected: SEM-001/004/007, FND-003. [D1](#), [D3](#)

6. The sandbox needs stronger isolation: qualified agreement. Require an explicit threat model and an exercised isolation boundary. A malicious repository is a different concern from a trusted repository containing restricted data. A sandbox CVE is evidence to inspect versions and mechanisms, not evidence that every current configuration is compromised. The four-escape count was not re-audited here. Affected: SAF-007, OPS-001/003. [D7](#)

7. Streamed traffic generally has no served-identity source: disagree with the cited basis. LiteLLM issue #7249 was fixed by PR #7263 in December 2024. Keep the receipt contract and test the exact deployed streaming/fallback combinations. Report measured unconfirmed traffic instead of assuming a near-total failure. Affected: OPS-006, RTG-008, TRU-002. [D4](#)

Dispositions continued

8. Failure-rate metrics miss a silently disabled gate: agree. Add active probes whose expected result is a block or pin plus an audit event. A configuration report and zero error counters do not show that a gate is on the actual path. Probe the enforced deployment without putting a real credential at risk. Affected: OPS-008, FND-005. This is an engineering conclusion; it does not require the cited CVE to remain open.

9. Secret detection must become regex plus an LLM: qualified agreement. Broaden the detector comparison and measure it on your traffic. Do not choose the mechanism by publication date or a cross-domain F1 score. Select an operating point that reflects the different costs of missed secrets and false pins. Affected: SAF-003/008, OPS-005. [S16](#), [S17](#)

10. Erasure and embedding risks require redesign: qualified agreement. Keep prompt-derived data sensitive, inventory every copy, and test destruction and restore. Consider deletable payload storage. Do not describe a benchmark inversion rate as universal, or treat blockchain-specific guidance as a blanket ruling on every append-only ledger. Affected: MEM-001/005/007/010, OPS-004. [S18](#), [S19](#), [D6](#)

11. Failure attribution needs grader and handoff faults: agree with the direction. A failed test, a broken verifier and a harmful handoff should not all teach the router that the chosen model lacks capability. Add attribution uncertainty and provenance before expanding categories. Preserve unresolved cases. More labels in an enum do not themselves establish causality. Affected: CAS-002, MEM-003/004, LRN-002. [S5](#), [S12](#)

12. Monotone pins need recovery and upward information-flow rules: qualified agreement. Mediate child returns and measure false blocking. Keep restrictions unless a trusted mechanism justifies a narrower release. APPA is a useful design reference, not permission to infer that a model-written summary is safe. Affected: SEM-006, SAF-001/002, CAS-008. [S15](#)

13. Evaluation gates need stronger statistics and a controlled pilot: agree, with corrections. Select deployable baselines before final evaluation; quantify how many comparisons distinguish the policies; randomize a constrained pilot. EVL-004 already includes representativeness and suppression, so it is not merely a label-count gate. D3 is observation of plumbing and behavior, not proof of savings. Affected: EVL-001/002/004/006/007. [S9](#), [S10](#), [S11](#)

14. Long-lived keys are dated and artifacts are unsigned: disagree with the blanket diagnosis. The implementation signs and verifies the manifest plus evaluation-report hash in `learning/artifacts.py`. Extra provenance and independently witnessed checkpoints may be useful. Their priority depends on who is trusted and what attack is being addressed. The availability of keyless signing does not, by itself, establish that a managed signing key violates ADRL's needs. Affected: OPS-002/007, LRN-005, TRU-003.

The pattern is consistent: many recommendations identify real questions, but fewer establish their proposed answer. The original report's 55 CURRENT, 17 CONTESTED and 5 DATED entries sum correctly. They should not be presented as 55 independently validated design decisions. "Current" mixes direct experiments, vendor contracts and engineering analogy.

Gaps that deserve more attention

The next risks are my synthesis of the architecture and evidence, including issues not resolved by the earlier critique.

The system needs an explicit boundary for its privacy promise. A model-call proxy can gate traffic it receives. It cannot infer that no tool, subprocess, telemetry client or vendor-hosted service sent data elsewhere. Likewise, hooks may provide partial observations without complete coverage. Describe the claim as control and accounting of specified model paths, and document the additional controls required for a whole-machine claim. This directly affects FND-001, SAF-009 and TRU-003.

A pin cannot protect a secret that was never detected. FND-004 allows an unpinned scanner failure to proceed upstream with an unscanned marker. That may be an intentional availability policy for unrestricted work, but it limits the assurance. Restricted workloads need authenticated classification before the first dispatch. Detection, identity and fail-open behavior belong in the same explanation of residual exposure.

Evidence origin and evidence quality are different axes. The T1/T5 conflict described earlier can exclude the very pairs the estimator needs. A branch can be verified rigorously yet differ from organic usage; an organic result can be poorly verified. Keep both facts rather than collapsing them into one quality ladder. Resolve this contract across LRN-001/002/003 and EVL-002/003/006 before implementing the learning objective.

Capability and deployed reliability need separate scorecards. Excluding infrastructure and dialect failures may help isolate a model's reasoning ability. Those failures still cost the developer time and the organization money. Publish both the result conditional on a healthy, compatible run and the result across all eligible attempted tasks. Otherwise a local deployment with frequent timeouts can look attractive after its failures are filtered away.

More information does not always justify another model call. In my illustrative counterexample, extra context reveals that one model's expected advantage is either 0.1 or 0.9. Both favor the same choice. The information changes confidence without changing the best action. Exploration must change enough decisions to repay its cost. [S4](#)

Integration maturity needs its own evidence. The product table labels Claude Code L3 while the same report says hook ingestion is missing. It also uses L0 for both inline passthrough and hooks-only observation. Separate a harness's theoretical capability, an implemented adapter, and the tested depth. Codex's current custom-provider configuration supports Responses as its wire API, but an endpoint alone does not solve identity, stored state, compaction or handoff. [D8](#)

These gaps are addressable. They also show why a careful pilot is more informative than another broad claim that the register matches the frontier.

What would convince me ADRL works

I would evaluate the privacy and routing claims separately, then test their interaction. Every stage below is a proposed experiment, not a result already obtained.

First, establish what the system actually sees and enforces. Use one supported harness, named versions and a small set of repositories. Exercise restart persistence, tool-result secrets, subagent returns, compaction, unsupported content, scanner timeouts, gateway fallback and missing receipts. Publish a coverage table: enforced, observed only, or outside scope. Use synthetic secrets and controlled test destinations for attack fixtures.

Second, collect representative shadow evidence. Measure request classes, gate overhead, proposed route distribution, cache usage, missing identity, receipts and exclusions. Shadow traffic can show whether instrumentation and classification work. It cannot reveal the quality of the local answer when the request actually went to a frontier model.

Third, execute a bounded comparison. Choose tasks with meaningful, independently specified acceptance checks. Compare a preselected strong baseline, a simple cheaper baseline, the current heuristic, bounded cheap-first escalation and a progress-aware candidate. Include downshifting where the interface supports it. Start each branch from the same recorded state and give each arm its own isolated workspace and permitted tools. Repeat a subset to estimate run-to-run variability.

Keep the paid input, output, cache effects, failed attempts, verification and local hardware assumptions visible. Measure task acceptance and developer waiting separately from token spend. Include privacy blocks and infrastructure failures in the end-to-end accounting, even if the capability analysis reports them separately.

Fourth, run a controlled pilot only after those checks pass. Assign comparable work to the candidate policy and control, with assignment stable within a lineage. Pre-register the acceptable quality loss, minimum worthwhile saving, latency limit and security blockers. Determine sample size from the effect worth detecting, the observed disagreement rate and clustering; do not substitute "300 labels" for that calculation. Report insufficient evidence when the comparison is unresolved. [S9](#), [S10](#), [S11](#)

Finally, test whether learning adds value. Compare the graduated heuristic, a simple retrieval/statistics baseline, the proposed effect estimator and any direct policy on the same permitted population. Evaluate whether any gain survives time, repository, user and serving-configuration changes. A learned router that cannot beat a simpler policy after its own overhead has not earned deployment.

Three outcomes would each be useful:

- If controls work and routing saves money at acceptable quality, ADRL earns wider routing scope.
- If controls work but routing adds little, ADRL may still justify itself as a privacy and evidence layer.
- If coverage is too incomplete or the operational burden outweighs value, narrow the integration or move the necessary controls into the harness/runtime.

The evidence should be allowed to change the product, including reducing its scope. That is consistent with ADRL's own principle that component presence is not readiness.

How this review was conducted

This is a second-pass source and architecture assessment, dated 7 September 2026. I reviewed the two local reports, 77 decision statements across ten current buckets, relevant clauses and implementation excerpts, and all fourteen ranked findings. I independently checked primary papers and additional relevant work. Following the owner's question, I directly read the dedicated taxonomy overview and TRU rationale and added pages 3-4.

Published-artifact check. The research critique opened successfully in the browser. Its 77 per-decision bodies match the local Markdown after normalizing formatting, punctuation and case; all fourteen ranked finding titles also match. The main findings were read on the published page. This establishes correspondence, not independent validation of its sources. The product artifact still requires sign-in, so its local Markdown remains the reviewed version. I have not established that the inaccessible product artifact is identical. The accompanying source-check record preserves the URLs, hashes and comparison method.

The checks emphasized a paper's actual comparison, information available to the policy, evaluation population, outcome measure and stated limitations. I distinguished a theorem under assumptions from measured performance, and distinguished both from an engineering recommendation. The scientific references below are not all peer-reviewed publications; arXiv availability alone is not evidence of peer review. Versioned links identify the text used where available.

This review does not reproduce the papers' experiments, re-audit all 315 URLs, certify production security, or independently validate every vendor integration and incident count. The earlier reports' test counts and maturity statements are reported history, not tests rerun for this document. The working trees contain uncommitted material; this report reviews the files present rather than a clean release. No register decision or implementation was changed.

The strongest corrections are directly checkable: the Agent-as-a-Router baseline arithmetic, the Replay Gap's stated limits, the closed LiteLLM bug, the manifest signing code, MEM-008's conditional gate, and the T1/T5 contract conflict. Recommendations about pilot design, product positioning and what to prioritize are my judgment based on those checks.

Reading the evidence

S references are research papers. D references are official documentation, maintainer records or regulatory guidance. Documentation establishes a product contract or recommendation; it is not experimental validation of ADRL. Inline references link to these primary sources in the PDF and Markdown editions.

Research sources: routing and learning

- **S1. RouteLLM: Learning to Route LLMs with Preference Data.** 2024, revised 2025, v4. Consulted full-text version. Comparative-outcome routing has an established precedent; benchmark transfer is not enterprise deployment validation. [Paper](#)
- **S2. A Unified Approach to Routing and Cascading for LLMs.** 2024/2025. Read abstract and record. Useful grounding for the dependence of cascades on quality estimation; used only at that level here. [Paper record](#)
- **S3. The Routing Plateau.** 2026, v1. Consulted full-text version, evaluation and limitations. Fixed pools and single-shot routing limit generalization. [Paper](#)
- **S4. SWE-Router.** 2026, v1. Consulted full-text version and proof. Partial-trajectory information can improve ideal decisions; acquisition cost and estimation remain practical questions. [Paper](#)
- **S5. The Handoff Tax.** 2026, v1. Consulted full-text version, design and limitations. Large controlled study of two model pairs; fixed switching and pricing assumptions matter. [Paper](#)

Evidence references continued

- **S6. The Replay Gap.** 2026, v1. Consulted full-text version and limitations. Strong methodological warning from a small, low-resolution pilot; routing-direction hypotheses remain unconfirmed there. [Paper](#)
- **S7. Agent-as-a-Router.** 2026, v3. Consulted full-text version, baseline comparison and limitations. The 15.3% statistic is relative to Vanilla, not the statistics-based heuristic. [Paper](#)
- **S8. Learning to Route LLMs from Bandit Feedback: One Policy, Many Trade-offs.** 2025, v1. Consulted full-text version and limitations. Preference-conditioned direct policy; static-log training and single-step scope. [Paper](#)
- **S9. Opportunity Is Not Realizability.** 2026, v1. Consulted full-text version, experimental design and limitations. Separates hindsight opportunity from deployable routing and addresses comparator selection. [Paper](#)
- **S10. Resolution Diagnostics for Paired LLM Evaluation.** 2026. Read abstract and record. Used for the need to assess paired-test resolution, not to prescribe a universal sample size. [Paper](#) [record](#)
- **S11. Position: Don't Use the CLT in LLM Evals With Fewer Than a Few Hundred Datapoints.** 2025, v3. Read abstract and record. Warns about small-sample uncertainty; it is not a universal theorem about every bootstrap. [Paper](#) [record](#)
- **S12. SlopCodeBench.** 2026, v1. Consulted full-text version and evaluation structure. Iterative quality erosion motivates observing more than a single passing test. [Paper](#)
- **S13. Defeating Prompt Injections by Design (CaMeL).** 2025, v1. Consulted full-text version and limitations. Structural control has a strong rationale and substantial integration assumptions. [Paper](#)
- **S14. Securing AI Agents with Information-Flow Control.** 2025, v1. Consulted full-text version and formal model. Relevant to mediating information flow rather than trusting agent instructions. [Paper](#)
- **S15. APPA: Recoverable Information-Flow Control for Real-World LLM Agents.** 2026, v2. Consulted full-text version, results and trusted-component assumptions. v1 has a different title and different results; this review uses v2. [Paper](#)
- **S16. Secret Breach Detection in Source Code with Large Language Models.** 2025, v1. Consulted full-text version and threats to validity. Supports evaluating contextual classification; public-repository contamination and workload transfer remain concerns. [Paper](#)
- **S17. Checked-In Secret Detection: Strings Are All You Need.** 2026. Consulted PDF method and limitations. A distinct detector candidate, with string-extraction and source-availability assumptions; no live ADRL result. [Paper](#)
- **S18. Ghost Vectors.** 2026, v1. Consulted full-text version and limitations. Soft-deleted HNSW data can persist; reconstruction is data- and model-dependent. [Paper](#)
- **S19. Zero2Text.** 2026. Read abstract and record. Supports treating textual embeddings as potentially invertible, not universal recovery of every embedding. [Paper](#) [record](#)

Documentation and local evidence

- **D1. Anthropic compaction documentation.** Checked the server-side block and continuation contract. [Documentation](#)
- **D2. Anthropic thinking documentation.** Checked model/prefix binding, account-date conditions and transformation reporting. [Documentation](#)
- **D3. Claude Code gateway compatibility guide.** Checked safety-classifier attribution and version-dependent behavior. [Documentation](#)
- **D4. LiteLLM issue #7249 and PR #7263.** The issue was opened in December 2024; the PR merged on 17 December 2024 and explicitly fixes the streaming API-base header. [Issue](#) | [Merged fix](#)
- **D5. LiteLLM Ollama provider documentation.** Includes proxy configuration targeting a local Ollama endpoint. [Documentation](#)
- **D6. EDPB Guidelines 02/2025, final v2, July 2026.** Read the final guidance, especially paragraphs 102-104. Blockchain-specific recommendations require an applicability assessment for ADRL. [Final guidance](#)
- **D7. Claude Code sandbox environments.** Recommends a dedicated VM for untrusted repositories and distinguishes isolation approaches. [Documentation](#)
- **D8. Codex configuration reference.** Checked the Responses-only custom-provider wire setting. [Official OpenAI documentation](#)

Local records used

- Product report, 3 September
- Research critique, 3 September
- Overview, design tenets, taxonomy and maturity and TRU rationale
- [Published research critique](#); artifact source-check record
- Decision index and the associated records under adr/
- MEM-008: Retrieval stays advisory until gated
- LRN-001: Evidence tiers
- LRN-002: Branched pairs
- LRN-003: Learning target
- EVL-004: Evidence gate
- EVL-007: Graduation
- Artifact signing and verification
- Gateway receipt attachment

The accompanying review manifest records hashes of the reports, register and implementation excerpts inspected. It identifies the reviewed working-tree contents without treating their existing commit as a complete snapshot.

ADRL's best-supported direction is to make model use more governable and its results more measurable. Its economic advantage should be earned through the same evidence discipline it proposes to enforce.